```
# Khanak Agrawal_03401222025_IGDTUW

# Part A: Understanding the Dataset
# Q1. Dataset Overview
# Load the dataset and answer the following:

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

df = pd.read_csv("agriculture_yield_dataset.csv")
```

```
# How many rows and columns are present?
print("No. of Rows:", df.shape[0])
print("No. of Columns:", df.shape[1])
```

```
No. of Rows: 1500
No. of Columns: 8
```

```
# What are the names of all columns?
df.columns
```

```
Index(['rainfall_mm', 'temperature_c', 'fertilizer_kg', 'irrigation_hours',
       'soil_ph', 'crop_type', 'soil_type', 'yield_ton_per_hectare'],
      dtype='object')
```

```
# Display the first 10 records.
df.head(10)
```

	rainfall_mm	temperature_c	fertilizer_kg	irrigation_hours	soil_ph	crop
0	588.6	18.6	242.4	6.5	6.5	
1	772.8	34.6	247.2	10.0	6.5	
2	970.9	36.3	168.4	7.3	6.4	
3	611.7	19.0	121.7	3.7	6.0	
4	696.1	29.6	184.6	5.1	6.1	
5	831.9	28.0	190.3	2.1	6.1	So
6	1023.8	32.0	108.5	6.9	6.2	
7	1142.4	18.4	241.9	4.1	7.3	So
8	810.4	36.4	164.8	9.9	6.5	
9	1085.5	29.4	89.3	8.3	5.8	

Next steps: Generate code with df | New interactive sheet

```
# Q2. Data Types and Missing Values
# Check the data type of each column.

df.dtypes
```

	0
rainfall_mm	float64
temperature_c	float64
fertilizer_kg	float64
irrigation_hours	float64
soil_ph	float64
crop_type	object
soil_type	object
yield_ton_per_hectare	float64

dtype: object

```
# Identify whether any missing values are present.

df.isnull().sum()
```

	0
rainfall_mm	0
temperature_c	0
fertilizer_kg	0
irrigation_hours	0
soil_ph	0
crop_type	0
soil_type	0
yield_ton_per_hectare	0

dtype: int64

```
# If missing values exist, mention the affected columns.

print(df.columns[df.isnull().any()])
```

```
Index([], dtype='object')
```

```
# Q3. Descriptive Statistics
# Generate summary statistics for all numerical features and answer:

print("\nSummary Statistics:")
print(df.describe())
```

```
Summary Statistics:
       rainfall_mm  temperature_c  fertilizer_kg  irrigation_hours  \
count  1500.000000    1500.000000    1500.000000       1500.000000
mean    754.054667      27.749467     148.744067          5.403267
std     255.097216       5.758101      56.990279          2.584329
min     300.200000      18.000000      50.300000          1.000000
25%     536.175000      22.600000      98.600000          3.200000
50%     761.200000      27.700000     146.850000          5.400000
75%     964.375000      32.600000     196.575000          7.600000
max    1200.000000      38.000000     249.900000         10.000000

           soil_ph  yield_ton_per_hectare
count  1500.000000            1500.000000
mean      6.759133               5.028793
std       0.719742               0.968282
min       5.500000               2.090000
25%       6.100000               4.337500
50%       6.800000               5.010000
75%       7.400000               5.740000
max       8.000000               7.860000
```

```
# Which feature has the highest mean value?
```

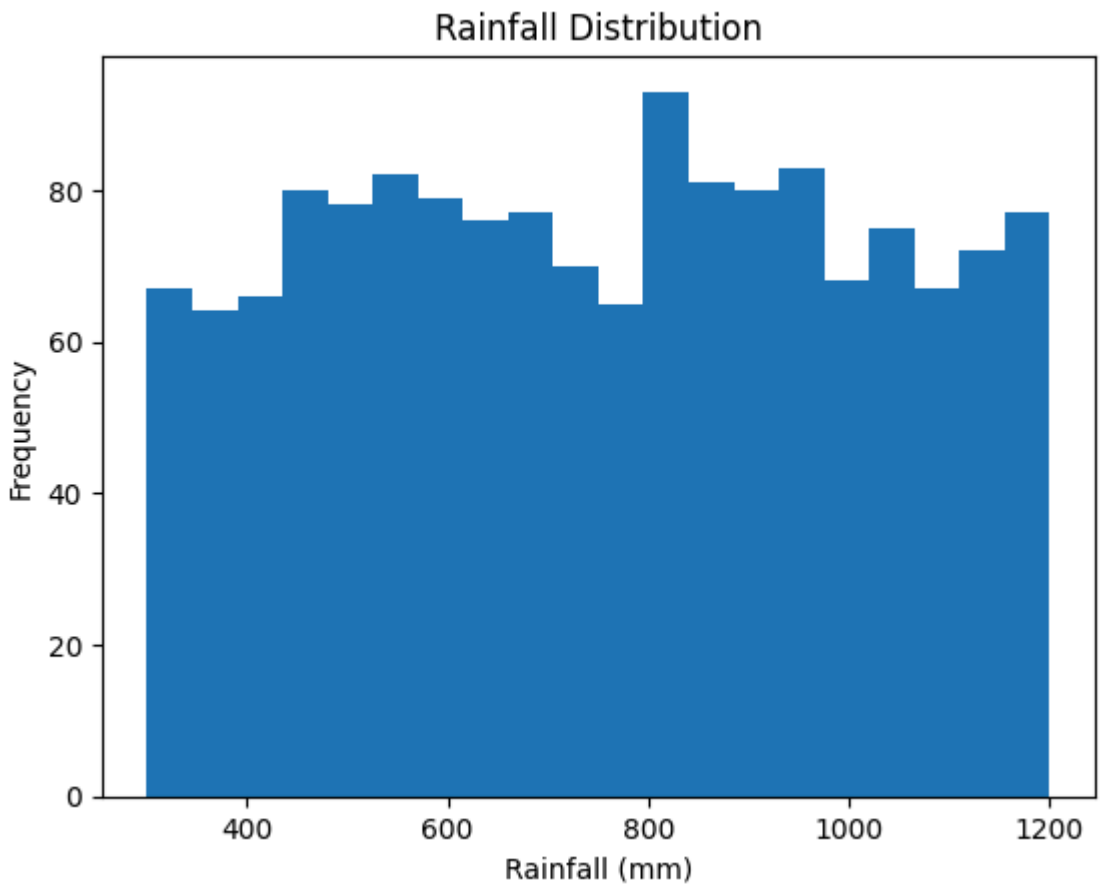
```
high_mean = df.mean(numeric_only=True)
print(high_mean.idxmax(), "=", high_mean.max())
```

```
rainfall_mm = 754.0546666666667
```

```
# Which feature has the highest standard deviation?

high_sd = df.std(numeric_only=True)
print(high_sd.idxmax(), "=", high_sd.max())
```

```
rainfall_mm = 255.0972161445094
```

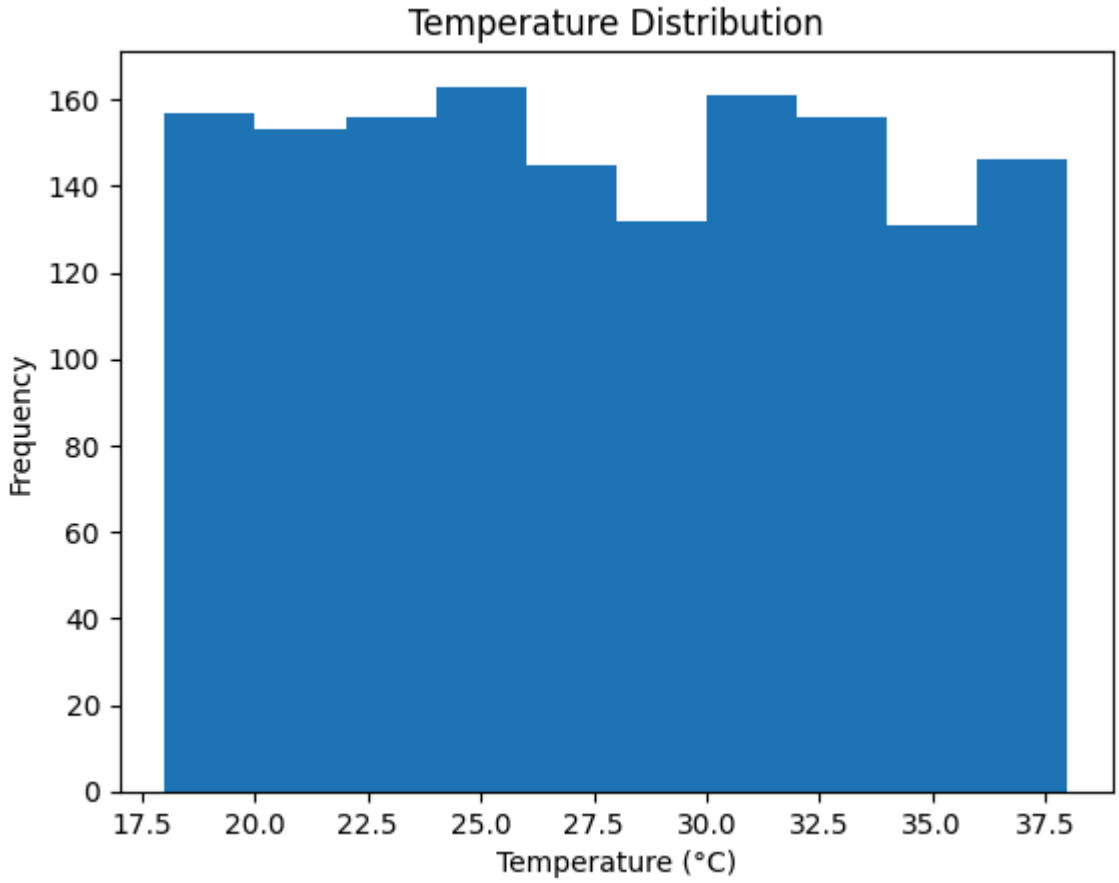
```
# Part B: Exploratory Data Analysis (EDA)
# Q4. Distribution Analysis
# Create histograms for:
# rainfall_mm

plt.hist(df['rainfall_mm'], bins = 20)
plt.title("Rainfall Distribution")
plt.xlabel("Rainfall (mm)")
plt.ylabel("Frequency")
plt.show()
```



Rainfall Distribution

- The rainfall values are spread across a certain range.
- Most of the data appears concentrated around the middle values.
- There are no extreme outliers, indicating balanced rainfall distribution.

```
# temperature_c

plt.hist(df['temperature_c'])
plt.title("Temperature Distribution")
plt.xlabel("Temperature (°C)")
plt.ylabel("Frequency")
plt.show()
```

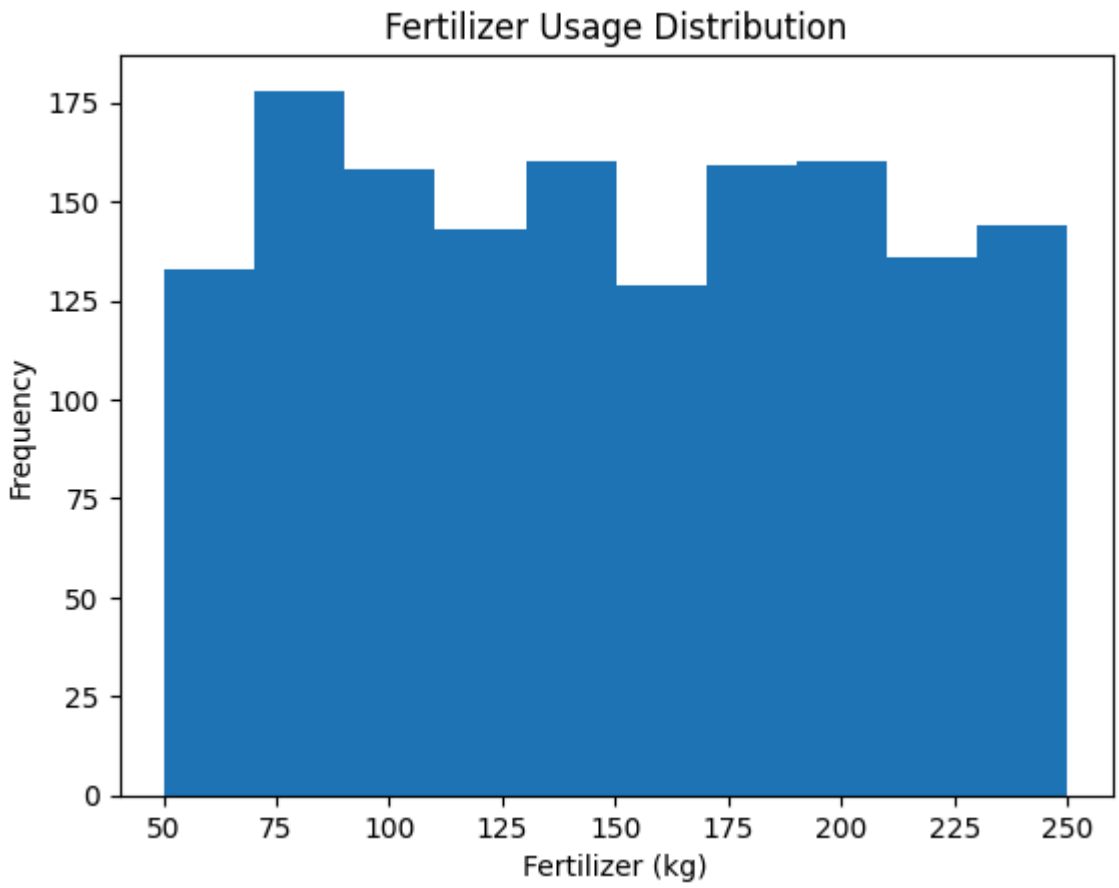


Temperature Distribution

- Temperature values show a fairly uniform/normal spread.
- Most regions fall within a moderate temperature range.
- No extreme skewness is observed in the data.

```
# fertilizer_kg

plt.hist(df['fertilizer_kg'])
plt.title("Fertilizer Usage Distribution")
plt.xlabel("Fertilizer (kg)")
```

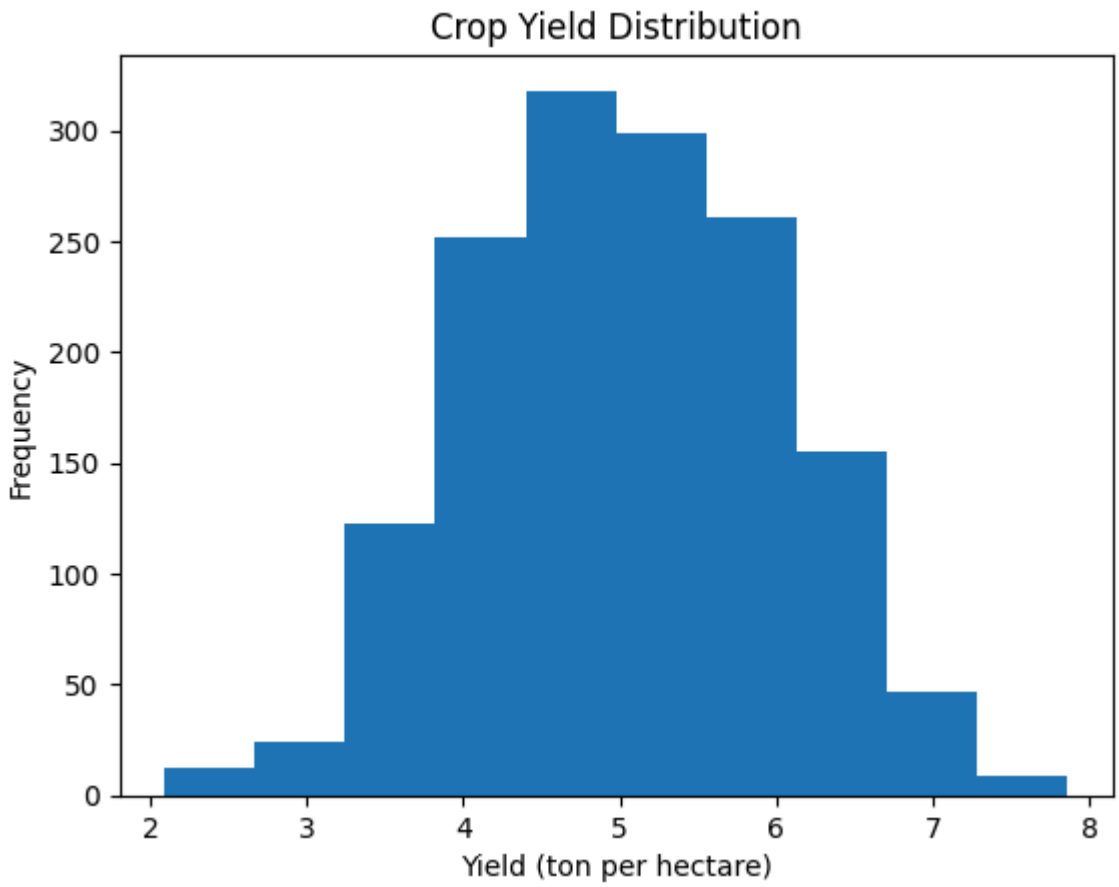
```
plt.ylabel("Frequency")
plt.show()
```



Fertilizer Usage Distribution

- Fertilizer usage is distributed across different levels.
- The majority of values are clustered around average usage.
- Very high or very low fertilizer usage is less frequent.

```
# yield_ton_per_hectare

plt.hist(df['yield_ton_per_hectare'])
plt.title("Crop Yield Distribution")
plt.xlabel("Yield (ton per hectare)")
plt.ylabel("Frequency")
plt.show()
```



Crop Yield Distribution

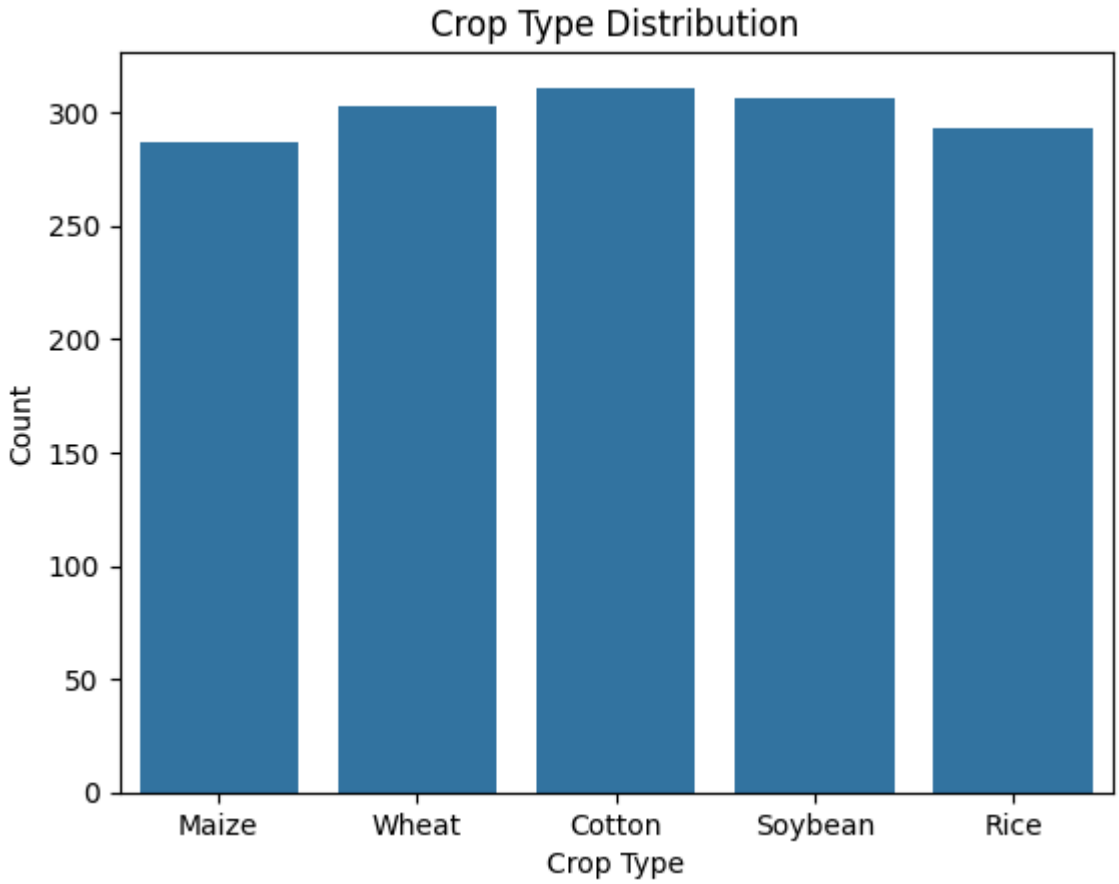
- Crop yield values are moderately distributed.
- Most farms achieve yields around the average range.
- There are few extreme outliers, suggesting stable agricultural output.

```
# Q5. Crop Type Analysis
# Find the number of records for each crop type.

df['crop_type'].value_counts()
```

crop_type	count
Cotton	311
Soybean	306
Wheat	303
Rice	293
Maize	287

dtype: int64

```
# Create a count plot (bar chart) for crop_type.

sns.countplot(x='crop_type', data=df)
plt.title("Crop Type Distribution")
plt.xlabel("Crop Type")
plt.ylabel("Count")
plt.show()
```




```
# Which crop appears most frequently?

df['crop_type'].value_counts().idxmax()
```

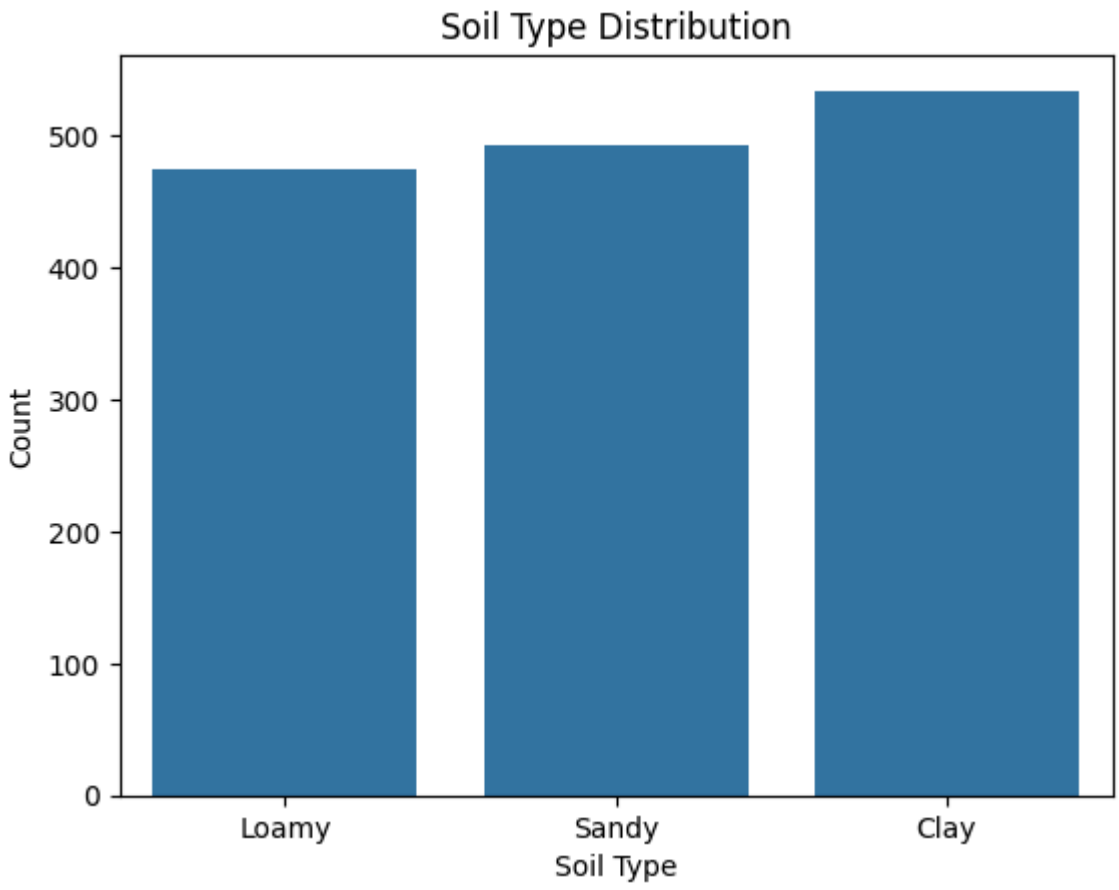
```
'Cotton'
```

```
# Q6. Soil Type Analysis
# Find the frequency of each soil type.

df['soil_type'].value_counts()
```

soil_type	count
Clay	534
Sandy	492
Loamy	474

dtype: int64

```
# Create a count plot for soil_type.

sns.countplot(x='soil_type', data=df)
plt.title("Soil Type Distribution")
plt.xlabel("Soil Type")
plt.ylabel("Count")
plt.show()
```

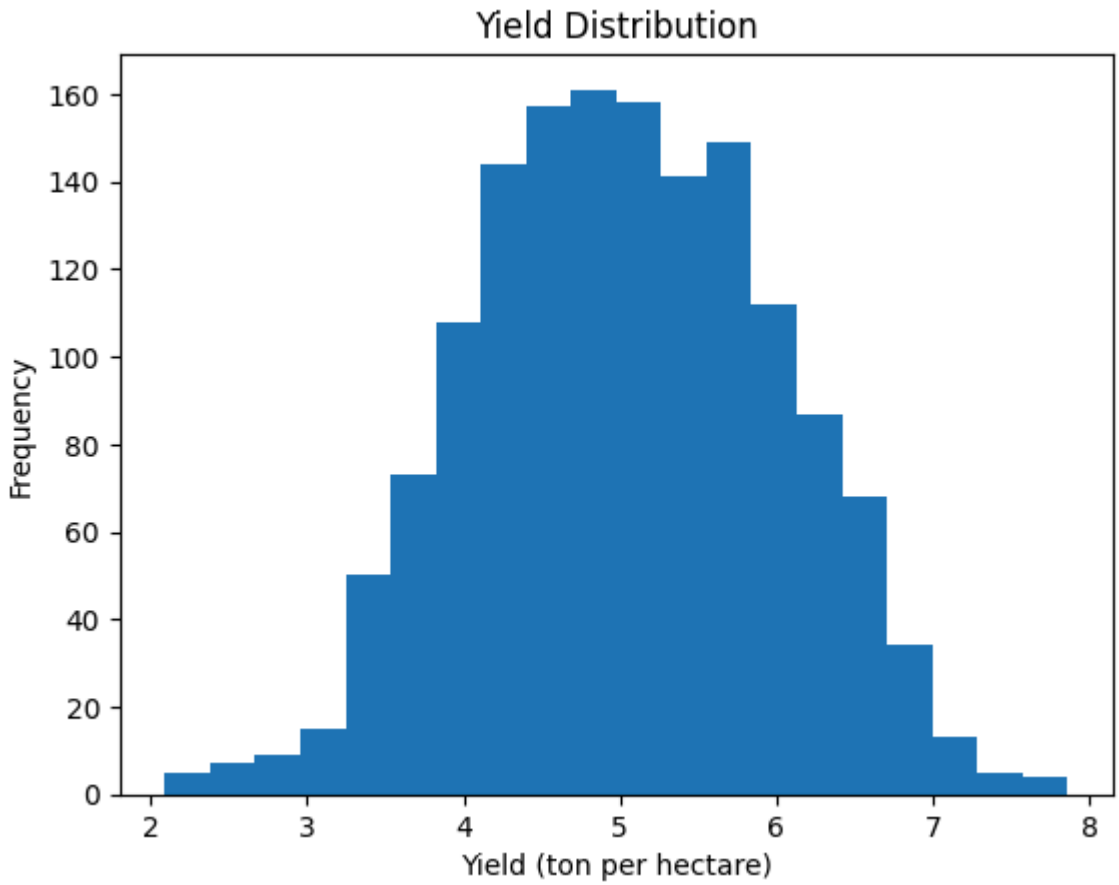



```
# Which soil type is most common?

df['soil_type'].value_counts().idxmax()
```

```
'Clay'
```

```
# Q7. Yield Distribution
# Create a histogram of yield_ton_per_hectare.
```

```
plt.hist(df['yield_ton_per_hectare'], bins=20)
plt.title("Yield Distribution")
plt.xlabel("Yield (ton per hectare)")
plt.ylabel("Frequency")
plt.show()
```



Is the distribution approximately normal?

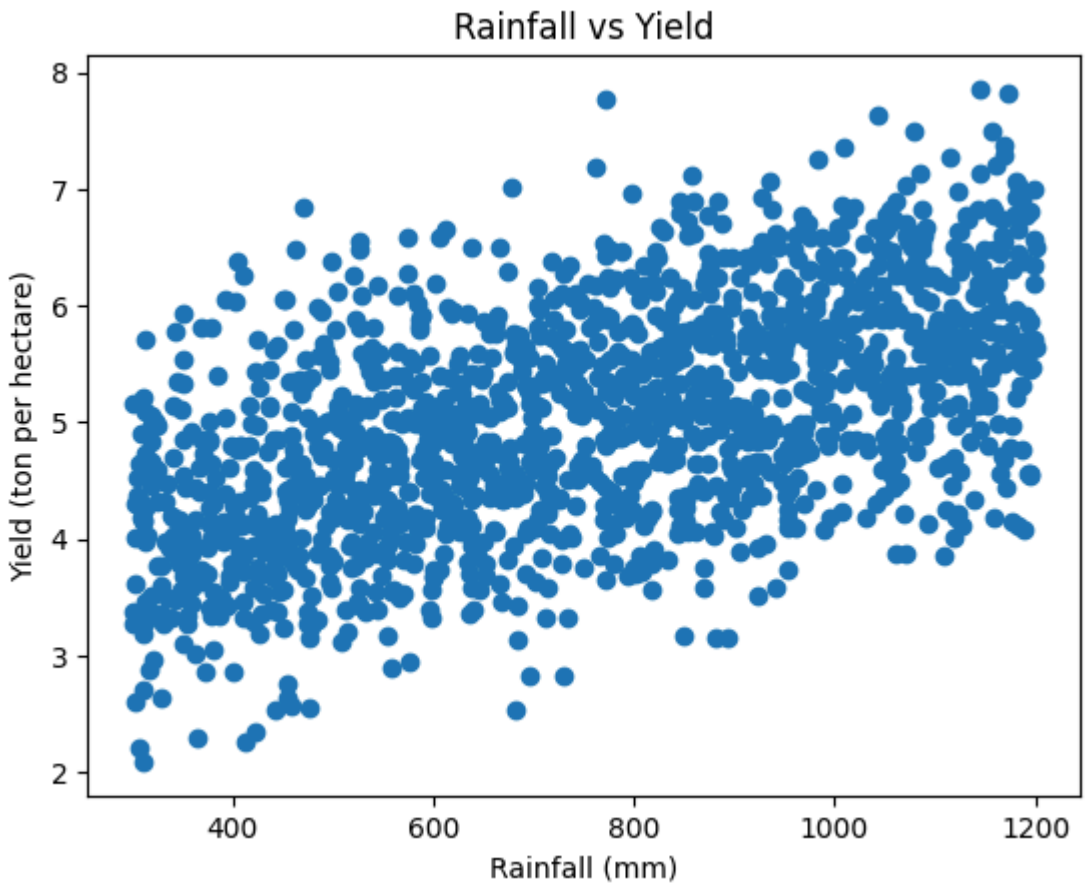
Ans: The distribution appears approximately normal as it is symmetric and follows a bell-shaped curve.

Are there any noticeable outliers?

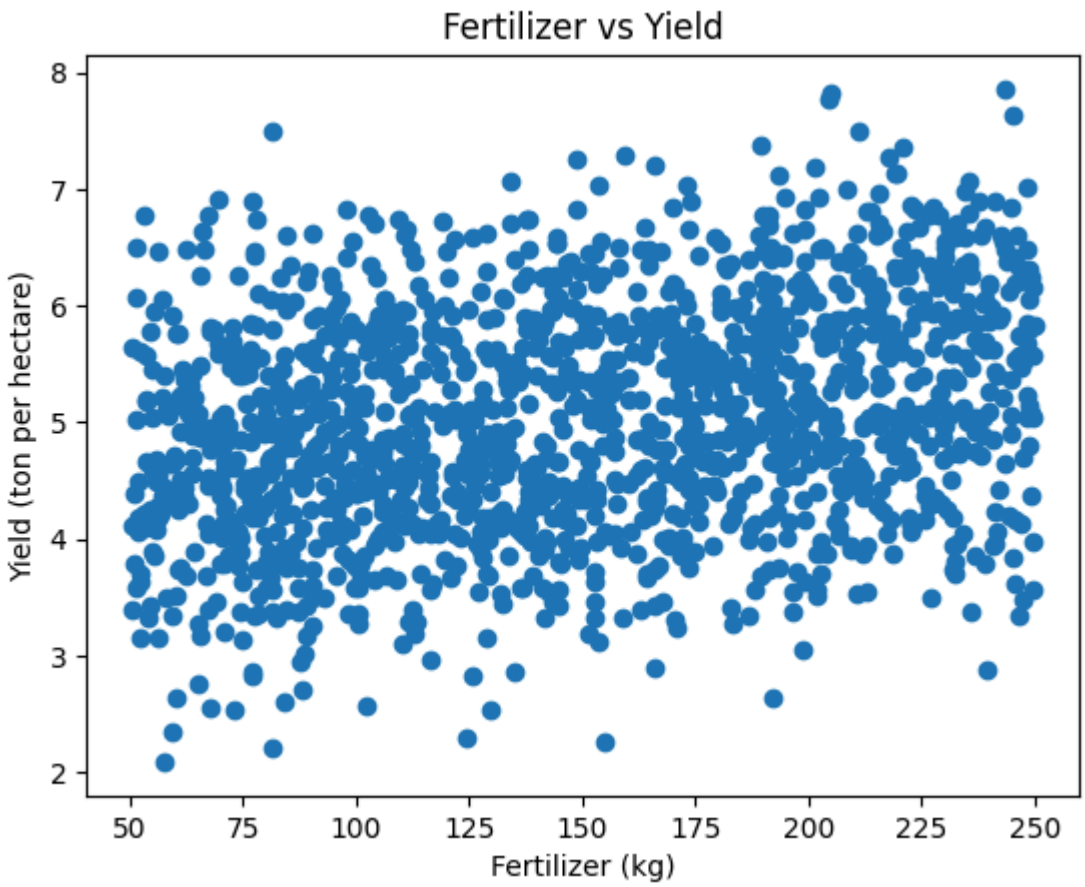
Ans: No significant outliers are observed, as most values are concentrated within a similar range.

```
# Q8. Scatter Plot Analysis
# Create scatter plots of:
# 1. rainfall_mm vs yield_ton_per_hectare

plt.scatter(df['rainfall_mm'], df['yield_ton_per_hectare'])
plt.title("Rainfall vs Yield")
plt.xlabel("Rainfall (mm)")
```

```
plt.ylabel("Yield (ton per hectare)")
plt.show()
```




```
# 2. fertilizer_kg vs yield_ton_per_hectare

plt.scatter(df['fertilizer_kg'], df['yield_ton_per_hectare'])
plt.title("Fertilizer vs Yield")
plt.xlabel("Fertilizer (kg)")
plt.ylabel("Yield (ton per hectare)")
plt.show()
```



Based on the plots: Which feature appears to have a stronger relationship with yield?

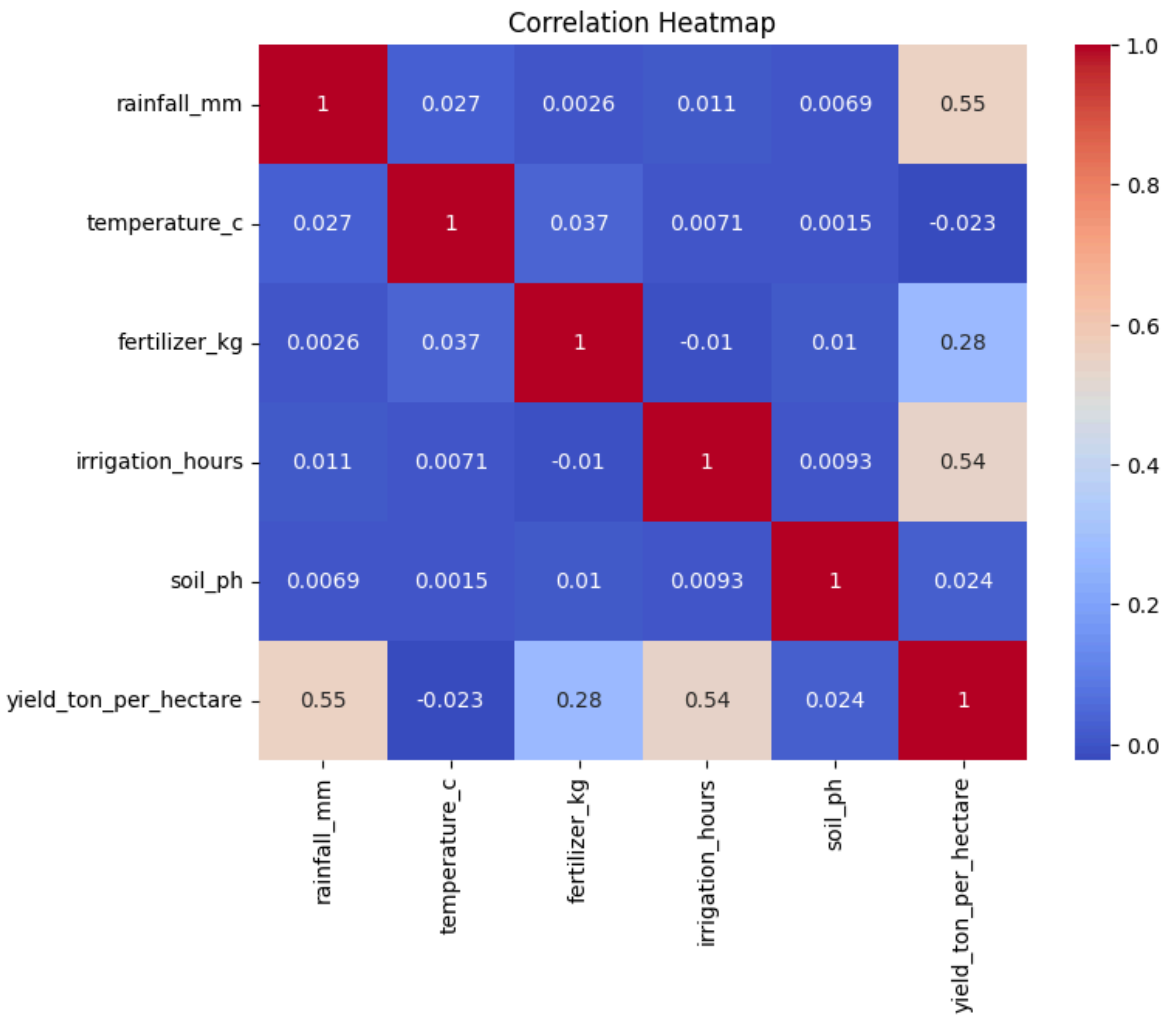
Ans: Both variables exhibit a positive relationship with yield, but rainfall shows a slightly stronger and more consistent trend. The fertilizer scatter plot is more dispersed, suggesting a weaker correlation. However, the difference between the two relationships is not very significant.

```
# Q9. Correlation Analysis
# Generate a correlation matrix for numerical features.

corr_matrix = df.corr(numeric_only=True)
corr_matrix
```

	rainfall_mm	temperature_c	fertilizer_kg	irrigation_h
rainfall_mm	1.000000	0.026721	0.002558	0.01
temperature_c	0.026721	1.000000	0.037468	0.00
fertilizer_kg	0.002558	0.037468	1.000000	-0.01
irrigation_hours	0.010877	0.007114	-0.010497	1.00
soil_ph	0.006916	0.001513	0.010001	0.00
yield_ton_per_hectare	0.553704	-0.022559	0.278043	0.54

Next steps: Generate code with `corr_matrix` New interactive sheet

```
# Create a heatmap.

plt.figure(figsize=(8,6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```




```
# Identify the top three features most correlated with crop yield.
```

```
corr_with_yield = corr_matrix['yield_ton_per_hectare'].sort_values(ascending
corr_with_yield
```

	yield_ton_per_hectare
yield_ton_per_hectare	1.000000
rainfall_mm	0.553704
irrigation_hours	0.542664
fertilizer_kg	0.278043
soil_ph	0.024412
temperature_c	-0.022559

dtype: float64

The top three features are rainfall_mm, irrigation_hours and fertilizer_kg

```
# Q10. Group-Based Analysis
# Calculate the average yield for:
# Each crop type

crop_avg = df.groupby('crop_type')['yield_ton_per_hectare'].mean()
crop_avg
```

crop_type	yield_ton_per_hectare
Cotton	4.607299
Maize	4.897143
Rice	5.494744
Soybean	5.173431
Wheat	4.989472

dtype: float64

```
# Each soil type

soil_avg = df.groupby('soil_type')['yield_ton_per_hectare'].mean()
soil_avg
```

soil_type	yield_ton_per_hectare
Clay	5.134326
Loamy	5.366519
Sandy	4.588882

dtype: float64

```
# Which crop and soil type have the highest average yield?

highest_crop = crop_avg.idxmax()
highest_crop_value = crop_avg.max()

highest_soil = soil_avg.idxmax()
highest_soil_value = soil_avg.max()

print("Highest Yield Crop:", highest_crop, highest_crop_value)
print("Highest Yield Soil:", highest_soil, highest_soil_value)
```

```
Highest Yield Crop: Rice 5.494744027303755
Highest Yield Soil: Loamy 5.366518987341772
```

```
# Part C: Data Preparation
# Q11. Feature Encoding
# The dataset contains categorical variables.
# Identify the categorical columns.

df.select_dtypes(include='object').columns
```

```
Index(['crop_type', 'soil_type'], dtype='object')
```

```
# Convert them into numerical form using One-Hot Encoding.

df_encoded = pd.get_dummies(df,
                            columns=['crop_type', 'soil_type'],
                            drop_first=True)
```

```
# Display the first five rows of the transformed dataset.

df_encoded.head()
```

	rainfall_mm	temperature_c	fertilizer_kg	irrigation_hours	soil_ph	yiel
0	588.6	18.6	242.4	6.5	6.5	
1	772.8	34.6	247.2	10.0	6.5	

Next steps: Generate code with df_encoded New interactive sheet

```
# Q12. Feature Selection
# Separate:
# Input features (X)
# Target variable (y)
# Specify which column is being used as the target variable.

X = df_encoded.drop(
```